

MODULE 7 · MULTI-TENANCY & SECURITY

OPA Gatekeeper

Policy enforcement at admission time — beyond what RBAC can check

RBAC decides who — Gatekeeper decides what

RBAC (Lab 7.2) only checks **identity and verb**: can this user create Pods in this namespace? It has no idea what's *inside* the object — no missing labels, no unapproved image, no privileged flag.

OPA Gatekeeper plugs that gap. It runs as an admission webhook that inspects every object's **content** before it's persisted, and blocks or flags the ones that don't comply.

BY THE END YOU CAN

- Explain admission control vs RBAC
- Tell a ConstraintTemplate from a Constraint
- Choose deny / warn / dryrun
- Exempt a namespace safely

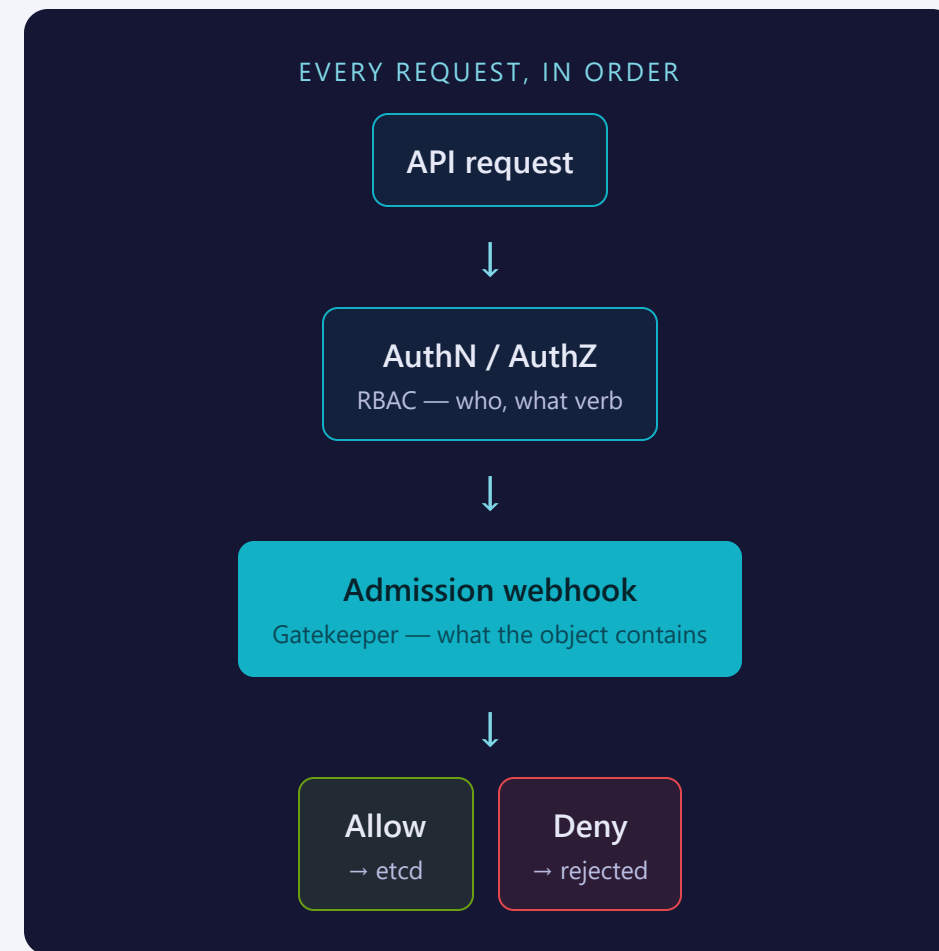
CONCEPT

Every write passes through admission before etcd

After authentication and authorization decide **who** may act, every create/update flows through the **admission chain** — the last checkpoint before the object is stored.

Gatekeeper registers itself as a **ValidatingWebhookConfiguration**: the API server calls out to it with the object, and it returns allow or deny.

This training cluster already has Gatekeeper installed and running in gatekeeper-system — the lab starts from writing policy, not installing the controller.



ConstraintTemplate defines it, Constraint applies it

	CONSTRAINTTEMPLATE	CONSTRAINT
Role	Defines the policy logic, written in Rego	Applies that logic with specific parameters
Written	Once per policy type	Once per enforcement (can have several)
Creates	A new CRD kind — e.g. <code>K8sRequiredLabels</code>	An instance of that CRD
Holds	<code>targets</code> (Rego) + the parameter schema	<code>match</code> (scope) + <code>parameters</code> + <code>enforcementAction</code>

MENTAL MODEL

ConstraintTemplate is the class; Constraint is the object — one template, many constraints (different namespaces, different parameters).

A full K8sRequiredLabels Constraint

```
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sRequiredLabels
metadata:
  name: require-pod-labels
spec:
  enforcementAction: deny      # deny | warn | dryrun
  match:
    kinds:
      - apiGroups:
          - ""
        kinds:
          - Pod
      namespaces:
        - gatekeeper-lab
    parameters:
      labels:
        - key: owner
          allowedRegex: "^[a-z][a-z0-9-]*$"
        - key: appName
          allowedRegex: "^[a-z][a-z0-9-]*$"
```

kind is the CRD its ConstraintTemplate generated; **match** scopes which objects/namespaces it applies to; **parameters.labels** lists each required key with its own allowedRegex.

Its ConstraintTemplate, in outline

template-required-labels.yaml (compact)

```
apiVersion: templates.gatekeeper.sh/v1
kind: ConstraintTemplate
metadata:
  name: k8srequiredlabels
spec:
  crd:
    spec:
      names:
        kind: K8sRequiredLabels # CRD kind used above
  targets:
    - target: admission.k8s.gatekeeper.sh
      rego: | # policy logic — labs/7.5/template-required-labels.yaml
```

The CRD kind under `crd.spec.names` must match the Constraint's kind on the previous slide; the Rego body lives in the referenced lab file.

Same pattern, different Rego

POLICY	CONSTRAINTTEMPLATE	CHECKS	DENIED EXAMPLE
Required labels	K8sRequiredLabels	owner + appName present, lowercase regex	Pod carrying only env: production
Allowed repos	K8sAllowedRepos	Image prefix matches an approved registry	Pod using registry.k8s.io/pause:3.9

THE ROLLOUT BRIDGE

Never launch a new policy straight into deny. Stage it: **dryrun** to measure impact → **warn** to notify teams → **deny** once they've remediated.

SEE IT

Three enforcement actions, three behaviours

ACTION	POD CREATED?	USER NOTIFIED?	VIOLATIONS RECORDED?	USE CASE
<code>dryrun</code>	Yes	No	Yes (audit only)	Impact assessment before rollout
<code>warn</code>	Yes	Yes — warning	Yes	Notify teams, give time to remediate
<code>deny</code>	No	Yes — error	Yes	Enforce in production

```
# deny – kubectl apply rejects it outright
Error from server: admission webhook denied the request:
you must provide labels: {"appName", "owner"}
```


Carve out exceptions without weakening the rest

METHOD	SCOPE	GRANULARITY	USE WHEN
<code>match.namespaces</code> allowlist	Per constraint	Only listed namespaces enforced	Piloting a new policy on one tenant
<code>match.excludedNamespaces</code> denylist	Per constraint	Everywhere except listed	Cluster-wide policy, named exceptions
<code>Config</code> resource	Cluster-wide	Exempt from all constraints + audit	System namespaces

ALWAYS EXCLUDE

kube-system and gatekeeper-system from cluster-wide policies — denying Gatekeeper's own Pods is a self-inflicted outage.

Where people trip

- **Deleting a ConstraintTemplate mid-lab.** It tears down the backing CRD — recreate a Constraint too soon and you'll hit "creation not allowed while custom resource definition is terminating."
- **Assuming Kubernetes' own validation is the gate.** K8s accepts mixed-case label values; Gatekeeper's allowedRegex is a separate, stricter check.
- **Treating dryrun as a no-op.** It never blocks, but it silently records violations for audit to surface later.
- **Forgetting namespace exclusions** on a cluster-wide constraint — Gatekeeper can end up denying its own controller Pods.

Commands to keep at hand

```
$ kubectl get pods -n gatekeeper-system      # controller + audit healthy?
$ kubectl get constrainttemplates            # rules installed
$ kubectl get constraints                    # every instance, any kind
$ kubectl describe k8srequiredlabels NAME    # match, parameters, violations
$ kubectl get config config -n gatekeeper-system -o yaml  # cluster-wide exemptions
```

Tip: describe the constraint, not just get — `status.violations` is where audit results land.

RECAP

Gatekeeper in one breath

- Admission control checks object **content** — RBAC only checks identity + verb
- ConstraintTemplate = the rule (Rego + CRD); Constraint = the instance (scope + parameters)
- dryrun (audit) → warn (notify) → deny (block)
- Exempt via `match.excludedNamespaces` (per constraint) or `Config` (cluster-wide)

HANDS-ON

Now run Lab 7.5
[labs/7.5/](#)

Next: [7.6 — Certificate Inspection](#)